Universidade Federal do Rio Grande do Norte
Departamento de Matemática e Informática Aplicada
DIM0438 - Redes de Computadores
Andriel Vinicius de Medeiros Fernandes

Sleep Token Guess Game

1. Introdução

A aplicação *Sleep Token Guess Game* consiste em um jogo simples *singleplayer*, onde o jogador deve identificar as músicas da banda britânica **Sleep Token** com base em trechos avulsos das letras, acumulando pontos quando acerta. A inspiração para este projeto veio da experiência anterior do autor no desenvolvimento de um jogo *multiplayer* de chat compartilhado com a linguagem Rust. Tal projeto, apesar de funcional e divertido, não utilizou de protocolos bem definidos, gerando dificuldades em sua implementação e uso; além disso, o autor tem bastante apreço pela banda em questão.

A princípio, o jogo possui 3 níveis de dificuldade:

- "Normal": dificuldade recomendada para iniciantes, imprime 1 linha por vez;
- "Fã": dificuldade mais avançada, imprime 1 linha por vez ocultando caracteres aleatórios;
- "Trve": dificuldade extrema, imprime 1 linha por vez sem nenhuma das vogais.

Para iniciar uma partida, o jogador deve informar seu nome de usuário bem como a dificuldade desejada para jogar. Antes do início da rodada, o jogo sorteia uma das músicas disponíveis e define como sendo a da rodada. A cada rodada, o jogador tem até 3 chances de palpites. O jogo sorteia uma linha da música definida e envia para o jogador; este, por sua vez, pode dar o seu palpite ou desistir imediatamente da partida, desconectando-se do jogo. Caso dê um palpite correto, o jogo computa $15 - (5 \times n)$ pontos no score, onde n é o número de tentativas usadas pelo jogador. Caso dê um palpite errado, o jogador pode receber uma nova linha da música e tentar novamente ou, se já tiver esgotado as tentativas, perder de vez o jogo. Ao conseguir adivinhar todas as músicas, o jogador ganha de forma definitiva o jogo. Nos cenários de derrota, desistência ou vitória, é exibido o total de pontos que o jogador acumulou até o momento.

O *Sleep Token Guess Game* foi projetado para oferecer suporte somente ao modo *singleplayer*. Contudo, isso não significa que só possa haver 1 cliente conectado ao servidor por vez: n clientes podem se conectar simultaneamente ao servidor e terão suas próprias sessões de jogo, sem nenhuma interferência entre si.

2. Características

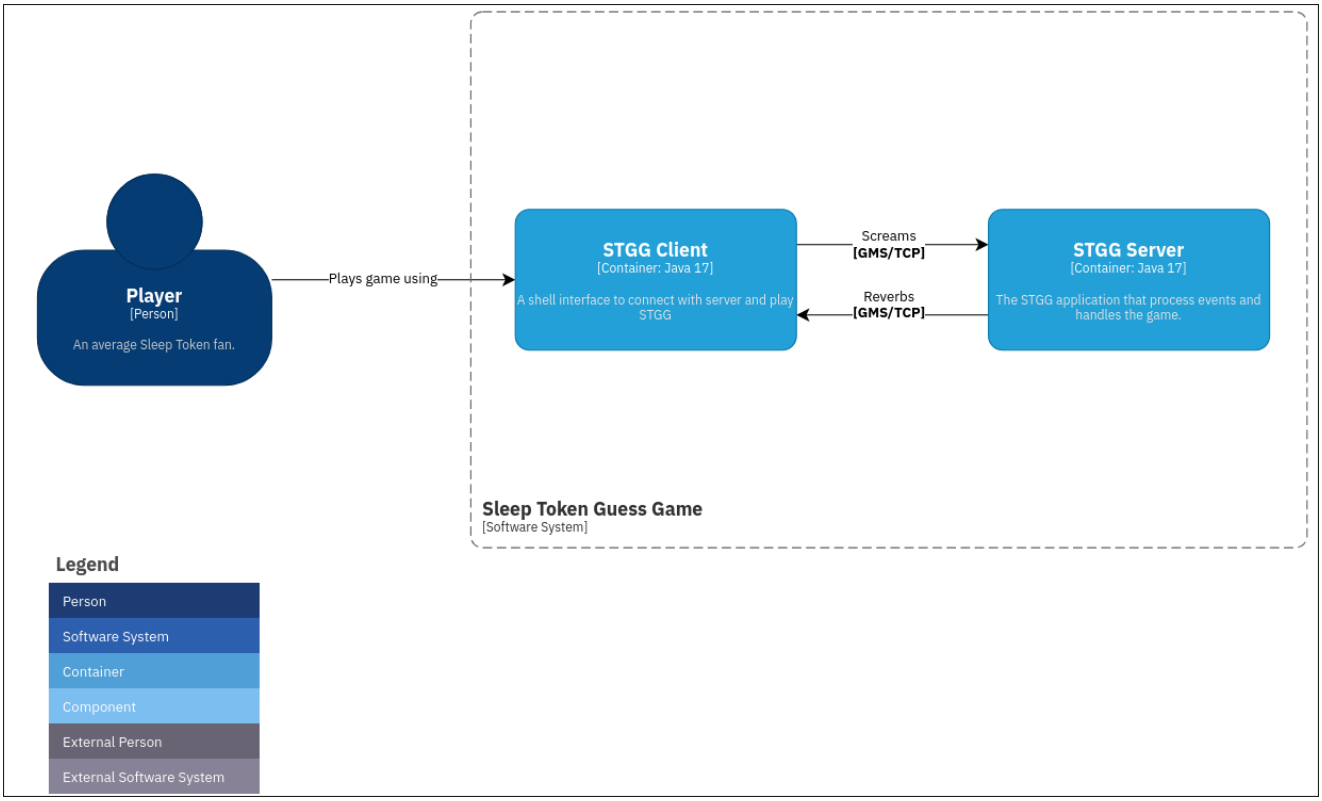
Pelo seu escopo simples, o jogo tem um conjunto limitado de funcionalidades:

- Jogador pode iniciar uma sessão com seu nome de usuário e dificuldade;
- Jogador pode dar palpites a cada rodada;
- Jogador pode desistir da partida enquanto ela decorre;
- Jogador pode solicitar ajuda para entender os comandos;
- Jogador pode ganhar o jogo ao descobrir todas as músicas;
- Jogador acumula pontos ao acertar.

Em termos de características de software, o jogo utiliza do protocolo *Guess Metal Song* (**GMS**) como protocolo de camada 7, além do protocolo *Transmission Control Protocol* (**TCP**) como protocolo de camada 4. A escolha do TCP decorre da necessidade da sessão de jogo ser contínua e integrada, não tolerante a interações perdidas na rede.

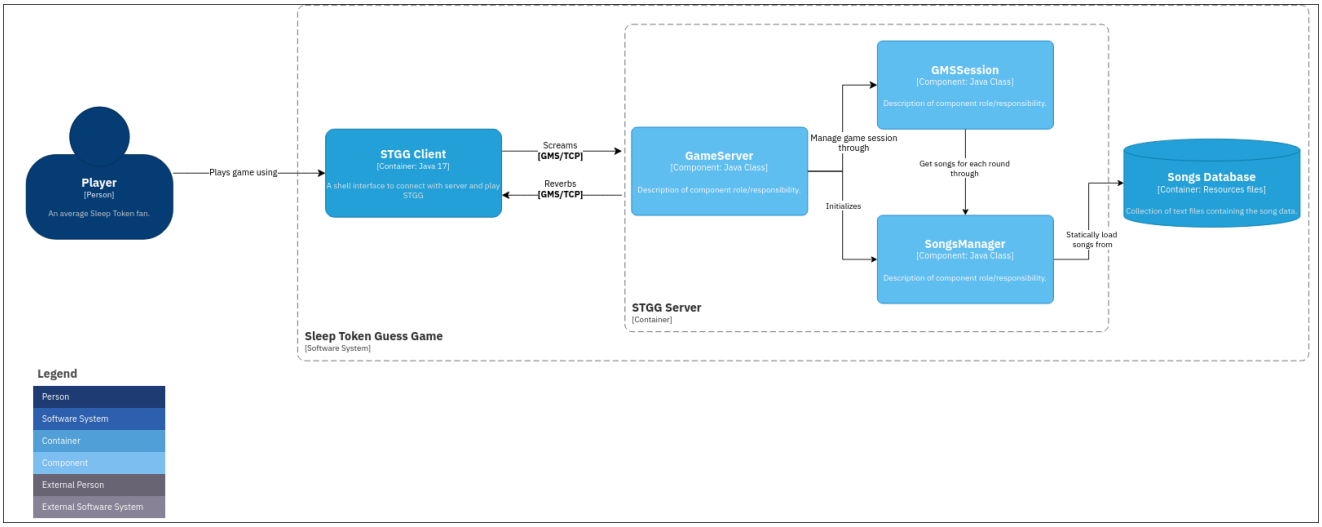
3. Arquitetura

Em relação à arquitetura do projeto, a Figure 1 apresenta em alto nível os componentes fundamentais do jogo:



Como é possível visualizar, um jogador deve acessar um cliente e este se conectar ao servidor. Com essa comunicação estabelecida, o jogo está pronto para acontecer.

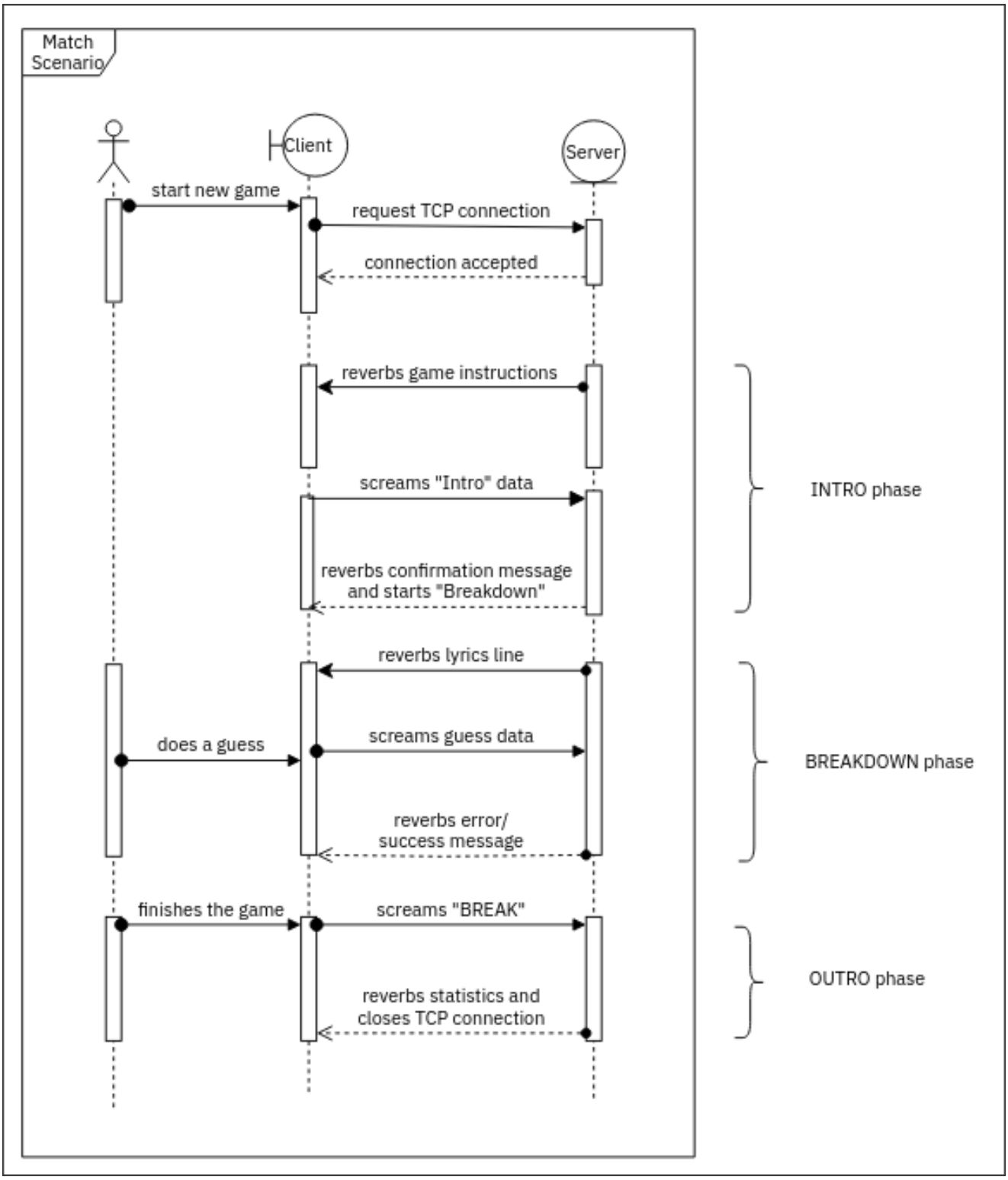
Na Figure 2 há uma visão mais detalhada do funcionamento do servidor:



De acordo com a figura, a classe que instancia o servidor relaciona-se com outras duas. A primeira é a classe responsável por lidar com a sessão de jogo do usuário, onde para cada cliente conectado com o servidor há uma nova instância dessa classe. Ou seja, uma nova sessão é criada para cada novo cliente. A segunda é a classe que carrega e mantém as músicas na memória, de forma *static* para que independente de sessão todas tenham acesso *read-only* às mesmas músicas. O servidor, ao ser criado, inicializa esta classe 1 única vez e sua instância é usada durante todo o programa.

As músicas carregadas para o programa estão armazenadas dentro do diretório de projeto `src/main/resources/songs`, diretório referido como *Songs Database*.

Na Figure 3 é apresentado o diagrama de mensagens do protocolo, incluindo as fases da comunicação.



4. Protocolo de aplicação GMS

O Protocolo GMS foi desenvolvido inicialmente para o contexto deste jogo, permitindo uma maneira eficaz de comunicação entre cliente-servidor. O GMS é **Stateful**, armazenando informações de uma sessão de jogo do lado do servidor enquanto a sessão está ativa. O GMS foi particularmente inspirado no protocolo *Simple Mail Transfer Protocol* (**SMTP**) em razão de suas restrições, como o uso do protocolo TCP para transporte e a necessidade de manter um estado contínuo no servidor.

4.1. Tipos de mensagem

O nome GMS sugere uma ligação direta com a temática do *heavy metal*. Durante um show, é comum que o público acompanhe as músicas cantando (ou gritando, a depender da plateia). Do mesmo modo, é esperado que o som produzido pelos instrumentos envolvidos ecoe e reverbere pelo ambiente, proporcionando prazer a quem escuta. Pensando nisso, os dois tipos de mensagens disponíveis na rede foram nomeados para referência com algum elemento desta temática:

- **Scream**: refere-se às mensagens enviadas pelo cliente ao servidor;
- **Reverb**: refere-se às mensagens enviadas do servidor ao cliente, independente se são fruto de mensagens do cliente ou se são independentes.

Dada à natureza e contexto do protocolo, os dados transferidos nas mensagens consistem em mensagens codificadas em ASCII. Não há suporte para blobs e outros tipos de mídia, embora possa haver uma extensão para suportar alfabetos diferentes e até emojis.

4.2. Fases da conexão

O GMS opera de forma Stateful em 3 fases diferentes, que também são nomeadas considerando a temática musical em questão:

- **INTRO**: fase em que o cliente acaba de estabelecer a conexão com o servidor. Neste ponto, o servidor envia um reverb com orientações ao cliente e aguarda sua resposta. Nesta fase só são permitidos os screams "INTRO" e "HELP", onde a única escolha do jogador é submeter suas informações ao servidor e iniciar a partida, avançando para o próximo estado;
- **BREAKDOWN**: fase onde a partida de fato ocorre. Aqui, o cliente é capaz de fazer palpites, acumulando pontos se acertar ou perdendo uma das 3 chances antes do jogo acabar. Nesta fase são permitidos os comandos "GUESS" e "BREAK", cabendo ao jogador permanecer tentando acertar ou desistir;
- **OUTRO**: fase onde a partida é finalizada e a pontuação é exibida. Aqui, não é possível nem necessária ação do cliente, já que ao chegar nesta fase a conexão entre cliente e servidor é fechada instantaneamente.

4.3. Comandos

4.3.1. Sintaxe

Acerca da sintaxe dos screams a gramática livre de contexto abaixo define como são formados:

- SCREAM -> CMD
- CMD -> INTRO | GUESS | BREAK | "HELP"
- INTRO -> "INTRO" LONGID
- GUESS -> "GUESS" LONGID
- BREAK -> "BREAK" OPTIONALID
- ID -> CHAR WORD | CHAR
- LONGID -> ID LONGID | ID
- OPTIONALID -> ID OPTIONALID | λ
- CHAR -> 'a' | 'b' | … 'z' | 'A' | 'B' | … 'Z' | '?' | '!' | '.' | '-' | ':' | ';' | '(' | ')' | '[' | ']'
- NUMBER -> 0 | … | 9
- WORD -> CHAR WORD | NUMBER WORD | CHAR | NUMBER

O estado inicial é definido como sendo "SCREAM". Para exemplificar os tipos de screams aceitos, tem-se:

```
INTRO vinicius hard
GUESS a blaze in the northern sky
GUESS Rom 5:8
BREAK
BREAK show score
HELP
```

Exemplos de screams incorretos seriam:

```
INTRO
GUESS
HELP help me please
```

4.3.2. Significado

Como visto na gramática acima, o usuário pode interagir com o protocolo enviando até 3 tipos de screams:

4.3.2.1. INTRO

O scream *INTRO* é responsável por comunicar ao servidor as informações do cliente, estas que são específicas de cada aplicação. Em razão da variedade de informações enviadas, o scream pode aceitar de 1 até n argumentos. Exemplos de argumentos seriam: nome de jogador, nome da banda, dificuldade, quantidade de pontos por rodada, entre outros.

No jogo em questão, as informações enviadas pelo scream são, obrigatoriamente, o nome do jogador e a dificuldade. Estes argumentos são processados pelo parser do jogo: se não houver exatamente estes 2 parâmetros, o scream é recusado. Além disso, a dificuldade deve ser uma dentre a lista apresentada na Introdução.

4.3.2.2. GUESS

O scream *GUESS* é responsável por comunicar ao servidor o palpite do cliente. O palpite deve consistir no título da música, composto por letras, dígitos e caracteres especiais ASCII.

No jogo em questão a implementação segue exatamente o padrão do GMS, não havendo detalhes relevantes a serem abordados.

4.3.2.3. BREAK

O scream *BREAK* é responsável por interromper a partida e finalizar o jogo. O scream aceita argumentos para trazer flexibilidade às aplicações.

No jogo em questão a implementação segue o padrão descrito mas com a ausência de argumentos. No cenário do jogo não é necessário passar parâmetros ao finalizar a partida, assim, o parser rejeita screams *BREAK* com argumentos.

4.3.2.4. HELP

O scream *HELP* é responsável por exibir orientações, lista de comandos disponíveis e outras informações definidas pela aplicação. Este scream não aceita argumentos.

No jogo em questão a implementação segue exatamente o padrão do GMS, não havendo detalhes relevantes a serem abordados.

5. Como executar

A aplicação foi desenvolvida inteiramente utilizando Java 17 e a ferramenta de compilação Gradle, então certifique-se de que para executar tens instalado o Java a partir dessa versão. Para rodar o jogo, é necessário habilitar primeiro o servidor e em seguida o(s) cliente(s), por meio dos comandos

```
./gradlew server
./gradlew client
```